Function Definition Compiler

A Python compiler front end: lexer, recursive-descent parser, AST and semantic analysis for function definitions, calls and return statements

GROUP 7 | TEAM MEMBERS

Ruth Mariam Joseph – 39

Ryan Jacob – 40

Teesa Thomas – 61

Thomas Peter – 64

Yash Bijo – 68

Zen Koshy – 69

Guide: Tintu Alphonsa Thomas
Batch: S5 CSE C

What Are We Building?

Beginners struggle to see how a compiler recognizes functions, parameters and expressions. We built a small front end that makes each stage visible.

1

Define the grammar

A CFG for functions, parameters, calls and expressions

2

Tokenize and parse

Lexer + recursive-descent parser validate input

3

Build the AST

A tree for every syntactically valid program

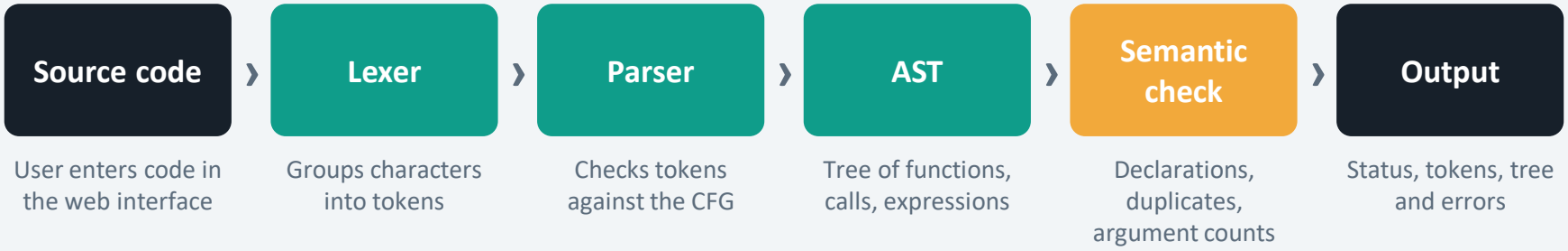
4

Check and report

Semantic checks plus lexical, syntax and semantic errors

Scope: an educational front end. It validates and represents programs; it does not execute them or generate machine code.

How the Front End Works



Syntax analysis (lexer + parser)

Is this a legal program? Catches missing parentheses, semicolons, end keywords and broken expressions.

Semantic analysis

Does it make sense? Syntax acceptance alone does not guarantee validity, so function and call rules are checked separately.

Context-Free Grammar

```
Program      → TopList
TopList      → Statement ; TopList | ε
Statement    → FunctionDef | Assignment | FunctionCall | PrintStmt
FunctionDef  → def id ( Params ) begin Body end
Body         → BodyStmt ; Body | ε
BodyStmt     → Assignment | FunctionCall | ReturnStmt | PrintStmt
ReturnStmt   → return Expr
Assignment   → id = Expr
PrintStmt    → print ( Expr )
FunctionCall → id ( Args )
Expr         → Term ExprTail
Term         → Factor TermTail
Factor       → id | number | FunctionCall | ( Expr )
```

Example input

```
def add(a, b) begin
  return a + b;
end;
```

Grammar notes

- Start symbol: Program
- Keywords: def, begin, end, return, print
- Operators: + - * /
- * and / bind tighter than + and -

Lexer, Parser and AST

lexer.py

Recognizes keywords, identifiers, numbers, operators and punctuation, then passes tokens on.

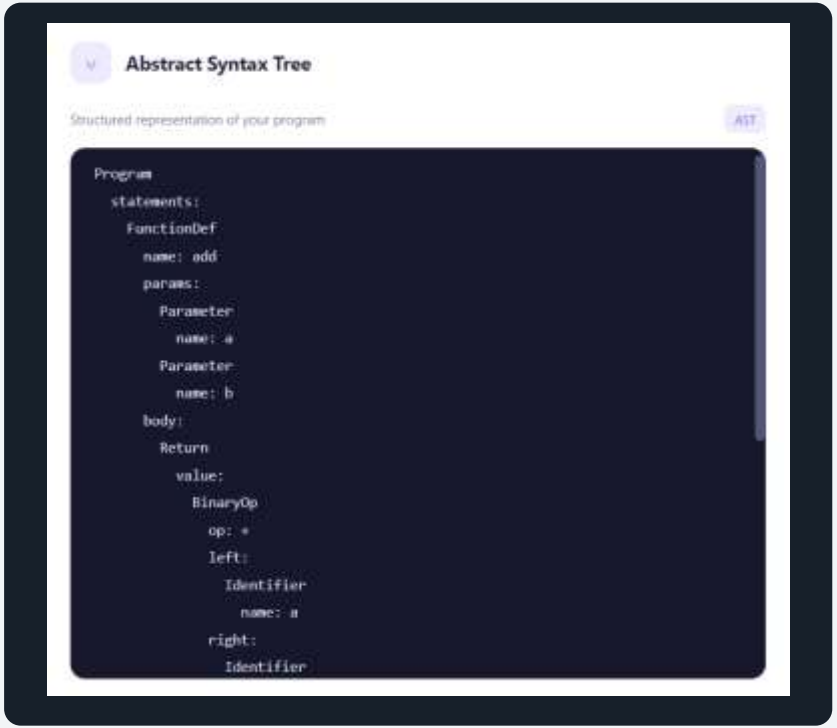
parser.py

Recursive descent over the CFG. Rejects missing parentheses, semicolons and incomplete expressions.

ast_nodes.py

Node structures for functions, parameters, calls, assignments and expressions.

*Precedence is built into the AST: for a + b * 2, multiplication sits below addition.*



AST output for def add(a, b)

Semantic Analysis

Rules targeted (semantic_analyzer.py)

- **Undefined function**

  `print(unknown(5));`

- **Duplicate parameter**

  `def add(a, a) …`

- **Argument count mismatch**

  `print(add(5));`

- **Duplicate function**

  `def greet() defined twice`

Verified in the demo

```
def add(a, b) begin
  return a + b;
end;
result = add(10);
```

```
Semantic analysis failed:
 function 'add' expects 2 argument(s), but got 1
```

Lexical and syntax stages pass; only the semantic stage fails.

Testing and Test Cases

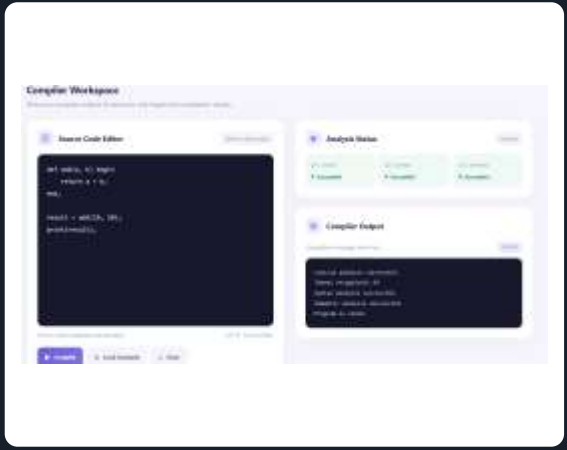
Valid programs accepted

- Simple function with print
- Parameters and return
- Arithmetic precedence
- Function calls inside print
- Multiple parameters and arguments
- Parenthesized expressions

Invalid programs rejected

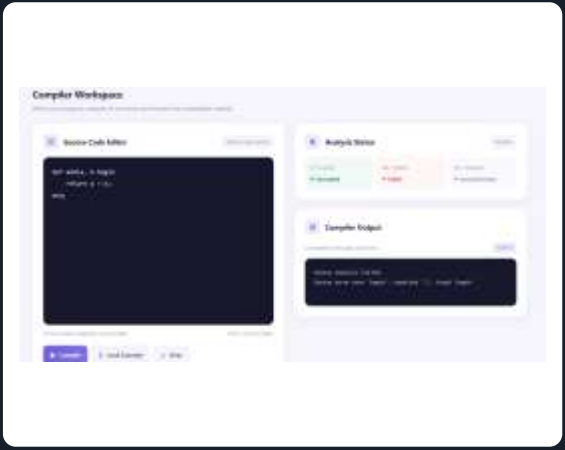
- Missing closing parenthesis
- Missing end keyword
- Missing semicolon
- Incomplete expression (a + ;)
- Invalid assignment (a = = 10)
- Semantic cases: undefined function, duplicate parameter, wrong argument count

Execution Results



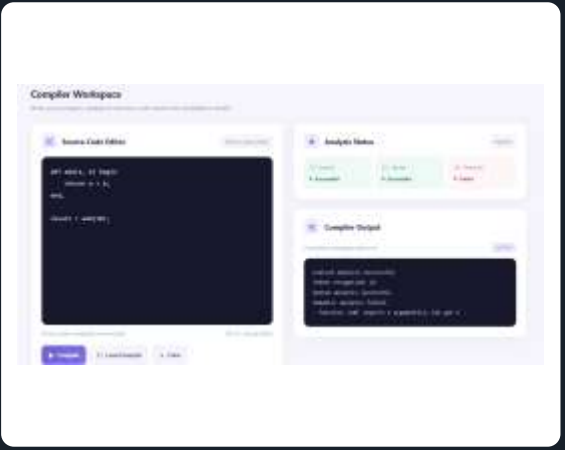
Successful compilation

All three stages pass



Syntax failure

Missing ")" in the header



Semantic failure

Argument count mismatch

The interface shows lexical, syntax and semantic status separately, plus tokens, the AST and compiler messages.

Conclusion

Lexer

Tokens for keywords, identifiers, numbers and operators

Parser

Validates functions, calls, statements and expressions

AST

Tree representation of the program structure

Semantic checks

Catches rule violations that syntax cannot